Here's a full detailed breakdown of every step in Phase 2.

Step 1 — Create the Patient entity

The entity is a Java class that maps directly to a database table. Every field becomes a column.

Create a class called `Patient` in a `model` package. Annotate the class with `@Entity` so JPA knows it maps to a table, and `@Table(name = "patients")` to name the table explicitly. Add `@Data` from Lombok to auto-generate getters, setters, equals, hashCode, and toString — no boilerplate.

Fields to add:

- `id` — type `UUID`, annotated with `@Id` and `@GeneratedValue(strategy = GenerationType.UUID)` so the database auto-generates a unique ID on insert
- `name` — type `String`
- `email` — type `String`
- `address` — type `String`
- `dateOfBirth` — type `LocalDate`
- `registeredDate` — type `LocalDate`, auto-set on creation

For `registeredDate`, use `@Column(nullable = false)` to enforce it at DB level. You can set a default in the service layer before saving.

Step 2 — Create the PatientRepository

The repository is an interface, not a class. You extend `JpaRepository<Patient, UUID>` — the first generic is the entity type, the second is the ID type.

Spring Data JPA reads this interface at startup and generates a full implementation automatically. You get `findAll()`, `findById()`, `save()`, `deleteById()`, and more — without writing a single line of SQL.

Add one custom method: `boolean existsByEmail(String email)`. Spring Data JPA reads the method name and generates the query — you don't write anything, just declare it. This is used later to check for duplicate emails.

Create this in a `repository` package.

Step 3 — Create the PatientService

The service layer contains all business logic. It sits between the controller (which handles HTTP) and the repository (which handles data). Controllers should never call the repository directly.

Create a class called `PatientService` in a `service` package. Annotate it with `@Service` so Spring registers it as a bean. Inject `PatientRepository` via constructor injection (not field injection — constructor injection is safer and easier to test).

Methods to implement at this stage:

`getAllPatients()` — calls `patientRepository.findAll()` and returns the list. Return type is `List<Patient>`.

`getPatientById(UUID id)` — calls `patientRepository.findById(id)`. This returns an `Optional<Patient>`. If the optional is empty, throw a `PatientNotFoundException` (you'll create this in step 10). If present, return the patient.

At this stage you don't need more methods — you'll add `createPatient`, `updatePatient`, and `deletePatient` in later steps.

Step 4 — Create the PatientController

The controller handles HTTP requests and delegates to the service. It should contain no business logic — only receive a request, call the service, and return a response.

Create a class called `PatientController` in a `controller` package. Annotate with `@RestController` (combines `@Controller` and `@ResponseBody`) and `@RequestMapping("/api/patients")` to prefix all routes. Inject `PatientService` via constructor.

Step 5 — Implement GET /patients and GET /patients/{id}

In `PatientController`, add two methods.

For `GET /patients`: annotate with `@GetMapping`, return type `ResponseEntity<List<Patient>>`. Call `patientService.getAllPatients()` and wrap in `ResponseEntity.ok(...)`. This returns HTTP 200 with the list as JSON.

For `GET /patients/{id}`: annotate with `@GetMapping("/{id}")`, add `@PathVariable UUID id` as the parameter. Call `patientService.getPatientById(id)` and return `ResponseEntity.ok(patient)`.

Start the application. Open Postman and hit `GET http://localhost:8080/api/patients`. You should get an empty list `[]` with a 200 status. The H2 database is empty at this point — that's fine.

Step 6 — Create the PatientRequestDTO

A DTO (Data Transfer Object) is a separate class that represents the shape of incoming request data. You never expose your entity directly to the outside world because:

- Entities have auto-generated fields like `id` and `registeredDate` that clients shouldn't be able to set
- Validation annotations belong on the DTO, not the entity
- The entity structure can change without breaking the API contract

Create `PatientRequestDTO` in a `dto` package. Fields are the same as the entity but without `id` and `registeredDate`:

- `name` — String
- `email` — String
- `address` — String
- `dateOfBirth` — LocalDate

Add `@Data` from Lombok. No JPA annotations — this is a plain Java object.

Step 7 — Implement POST /patients

In `PatientService`, add a `createPatient(PatientRequestDTO requestDTO)` method.

Inside the method:

1. Map the DTO to a `Patient` entity manually — create a new `Patient`, set each field from the DTO
2. Set `registeredDate` to `LocalDate.now()`
3. Call `patientRepository.save(patient)` — JPA persists it and returns the saved entity with the generated ID
4. Return the saved patient

In `PatientController`, add a method annotated with `@PostMapping`. The parameter is `@RequestBody PatientRequestDTO requestDTO`. Call `patientService.createPatient(requestDTO)` and return `ResponseEntity.status(HttpStatus.CREATED).body(patient)` — this returns HTTP 201 instead of 200, which is the correct status for resource creation.

Test in Postman: send a `POST` with a JSON body containing name, email, address, dateOfBirth. You should get a 201 response with the created patient including its generated UUID.

Step 8 — Add Bean Validation to the DTO

Add the Spring Boot Validation starter to your `pom.xml` if not already present.

On `PatientRequestDTO`, add validation annotations to each field:

- `name` — `@NotBlank(message = "Name is required")`
- `email` — `@NotBlank(message = "Email is required")` and `@Email(message = "Invalid email format")`
- `address` — `@NotBlank(message = "Address is required")`
- `dateOfBirth` — `@NotNull(message = "Date of birth is required")` and `@Past(message = "Date of birth must be in the past")`

In the controller, add `@Valid` before the `@RequestBody PatientRequestDTO requestDTO` parameter. Without `@Valid`, Spring will not run the validation annotations — they are silently ignored.

Test in Postman: send a POST with an empty body or a missing field. Right now you'll get a 400 error with a messy internal Spring error response. The next step fixes this.

Step 9 — Create the global exception handler

Without a custom handler, Spring returns its default `DefaultHandlerExceptionResolver` error format which is verbose and exposes internals. You want a clean, consistent JSON structure.

Create a class called `GlobalExceptionHandler` in an `exception` package. Annotate with `@ControllerAdvice` — this means Spring will apply it to all controllers.

Add a method annotated with `@ExceptionHandler(MethodArgumentNotValidException.class)`. This fires whenever `@Valid` fails. Inside the method:

1. Iterate over `ex.getBindingResult().getFieldErrors()`
2. Build a `Map<String, String>` where the key is the field name and the value is the error message
3. Return `ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errors)`

Now when you send a POST with missing fields, you'll get a clean response like:

```
{
  "name": "Name is required",
  "email": "Email is required"
}
```

Step 10 — Add custom exception classes

Spring returns 500 by default for any unhandled exception. You want specific HTTP codes for specific situations.

Create two exception classes in the `exception` package:

`PatientNotFoundException` — extends `RuntimeException`. Constructor takes a `String message` and passes it to `super(message)`.

`EmailAlreadyExistsException` — same structure.

In `GlobalExceptionHandler`, add two more `@ExceptionHandler` methods:

One for `PatientNotFoundException` — returns `ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", ex.getMessage()))`.

One for `EmailAlreadyExistsException` — returns `ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of("error", ex.getMessage()))`.

Now in `PatientService.getPatientById()`, throw `new PatientNotFoundException("Patient not found with id: " + id)` when the optional is empty — this will automatically produce a clean 404 response.

Step 11 — Implement PUT /patients/{id}

In `PatientService`, add an `updatePatient(UUID id, PatientRequestDTO requestDTO)` method.

Inside the method:

1. Fetch the existing patient using `getPatientById(id)` — this already throws 404 if not found
2. Check if the new email is different from the current email AND already exists in the database using `patientRepository.existsByEmail(requestDTO.getEmail())` — if so, throw `EmailAlreadyExistsException`
3. Update each field on the existing patient object from the DTO
4. Call `patientRepository.save(patient)` — JPA detects the existing ID and issues an UPDATE, not an INSERT
5. Return the updated patient

In `PatientController`, add a method annotated with `@PutMapping("/{id}")` with both `@PathVariable UUID id` and `@Valid @RequestBody PatientRequestDTO requestDTO` as parameters.

Test in Postman: update a patient's name — should return 200 with the updated data. Try updating to an email that already belongs to another patient — should return 409 Conflict.

Step 12 — Implement DELETE /patients/{id}

In `PatientService`, add a `deletePatient(UUID id)` method.

Inside: call `getPatientById(id)` first — this throws 404 if the patient doesn't exist, which is the correct behaviour (you can't delete something that isn't there). Then call `patientRepository.deleteById(id)`.

In `PatientController`, add a method annotated with `@DeleteMapping("/{id}")` with `@PathVariable UUID id`. Return `ResponseEntity.noContent().build()` — this is HTTP 204, the correct status for a successful deletion with no response body.

Test in Postman: delete an existing patient — 204. Try to delete a non-existent ID — 404 with the error message.

Step 13 — Add Springdoc OpenAPI

Add the `springdoc-openapi-starter-webmvc-ui` dependency to `pom.xml`.

No code changes needed. Springdoc scans your controllers and automatically generates an OpenAPI 3.0 specification from your annotations.

Start the app and open `http://localhost:8080/swagger-ui.html` in a browser. You'll see all your endpoints listed with their request/response schemas, HTTP methods, and status codes. You can execute requests directly from the UI.

Optionally, add `@Operation(summary = "...")` on each controller method and `@Tag(name = "Patient Management")` on the controller class to improve the documentation quality.

How the layers connect — the full request flow

When a client sends `POST /api/patients`:

1. Spring routes the request to `PatientController.createPatient()`
2. `@Valid` triggers Bean Validation on the `PatientRequestDTO` — if it fails, `GlobalExceptionHandler` catches the error and returns 400
3. The controller calls `PatientService.createPatient(requestDTO)`
4. The service checks for duplicate email using `PatientRepository.existsByEmail()` — throws `EmailAlreadyExistsException` if found, caught by the handler as 409
5. The service maps the DTO to a `Patient` entity and calls `PatientRepository.save(patient)`
6. JPA issues an `INSERT` to H2, auto-generates a UUID, and returns the saved entity

7. The service returns the entity to the controller
8. The controller wraps it in `ResponseEntity` with status 201 and returns it as JSON

Every layer has one job. Nothing bleeds into another layer. This is the foundation the rest of the system builds on.

Phase 2 steps implement complete guide

https://chatgpt.com/share/6a1890d6-afa0-8321-9942-18adbdafb2f7